Data Structure

Calpephore Nkikabahizi

OBJECTIVES

- **The course should enable the students to :**

○ Solve efficiently Problem

1. Develop **optimal solutions** to computational problems,
2. Use the right **data structures** to store, access, and organize data,
3. Design **algorithms** that are time- and space-efficient.

○ Improve Computational Efficiency

- Analyze performance using **Big-O notation**
- Reduce runtime and memory usage in software
- Understand trade-offs between **different approaches** to solving the same problem

Objectives

- **The course should enable the students to :**

- **Design System and Software**

1. Form the **core of operating systems, compilers, databases, and networks**.
2. Help design **modular, scalable, and maintainable** systems.

- Get Foundation for Advanced Topics (Ai, Machine learning , cryptography, network,..)

Prerequisite

- Calculus ,or General math
- Basic Algebra
- Statistical theories
- Probability
- Fundamental of algorithms
- Discrete mathematics

CONTENT

UNIT-I INTRODUCTION TO ALGORITHMS

Algorithms: Definition, Properties, presentation, algorithm structure, categories, decision structures, search and sort. We will also describe performance Analysis-Space Complexity, Time Complexity, Asymptotic Notations.

UNIT-II SORTING AND SEARCHING

- **Sorting:** Introduction, Selection sort, Bubble sort, Insertion sort, Merge sort, Quick sort, Heap Sort.
- **Searching:** Introduction, Linear search, Binary search, Fibonacci search.

CONTENT

UNIT-III INTRODUCTION TO DATA STRUCTURES

Data structures: Introduction, Data Structures types, DS Operations.

UNIT-IV ARRAY, LIST AND LINKED LIST

Arrays: Introduction, arrays categories, arrays operations: Applications**:** Data mining, image processing , robot motion ,chess game

List : Introduction, list operations: Applications**:**

Linked lists: Introduction, Singly linked lists, Circular linked lists, Doubly linked lists, Multiply linked lists, Applications: Web pages viewers,

CONTENT

UNIT-V STACKS AND QUEUES

Stacks: Introduction, Stack Operations, Applications: Infix to Postfix Conversion, Evaluation of Postfix Expression.

Queues: Introduction, Operations on queues, Circular queues, Priority queues. Implementation of Stack and Queue using linked list.

CONTENT'

UNIT-VI Set, **Graph, Trees and Hashing Table**

- **Trees:** Introduction, Definition and basic terminologies, Representation of trees.
- **Binary Trees:** Basic Terminologies and Types, Binary Tree Traversals, Binary Search Trees.

UNIT-VII Files

Introduction, Definition and operations

UNIT-VIII Projects

CHAP. 1-1 INTRODUCTION ALGORITHM

An algorithm

- An Algorithm is a finite sequence of instructions, a logic and explicit step by step procedure for solving a problem starting from a known beginning. The number of instructions must be restricted

The algorithm is different from a program,

- A program is a set of logically related instructions that is arranged in a sequence that directs the computer in solving a problem

CHAP. 1-1 INTRODUCTION ALGORITHM

- Algorithm has the following basic properties
  Input-Output:- Algorithm takes '0' or more input and produces the required output.
  Finiteness:- An algorithm must terminate in countable number of steps.
  Definiteness: Each step of an algorithm must be stated clearly and unambiguously.
  Effectiveness: Each and every step in an algorithm can be converted in to programming language statement.
  Generality: Algorithm is generalized one. It works on all set of inputs and provides the required output. In other words it is not restricted to a single input value.

CHAP. 1-2 ALGORITHM PRESENTATION

Tools used to convert algorithms into computer programs are:

- **Pseudocode** is a detailed yet readable description of what a computer program or algorithm must do, expressed in a formally-style natural language rather than in a programming language
- **Flowcharts** : Graphical representation of the logical steps to carry out a task show how the steps related to each other. A program flowchart shows the preprogram structure, logic flow and operations performed.
- **Write the Program (Coding)**

Coding involves the conversion of an algorithm to a program, written in any programming language. The programmer must know the syntax of the programming language chosen.

CHAP. 1-2 ALGORITHM PRESENTATION

Flowchart Symbols

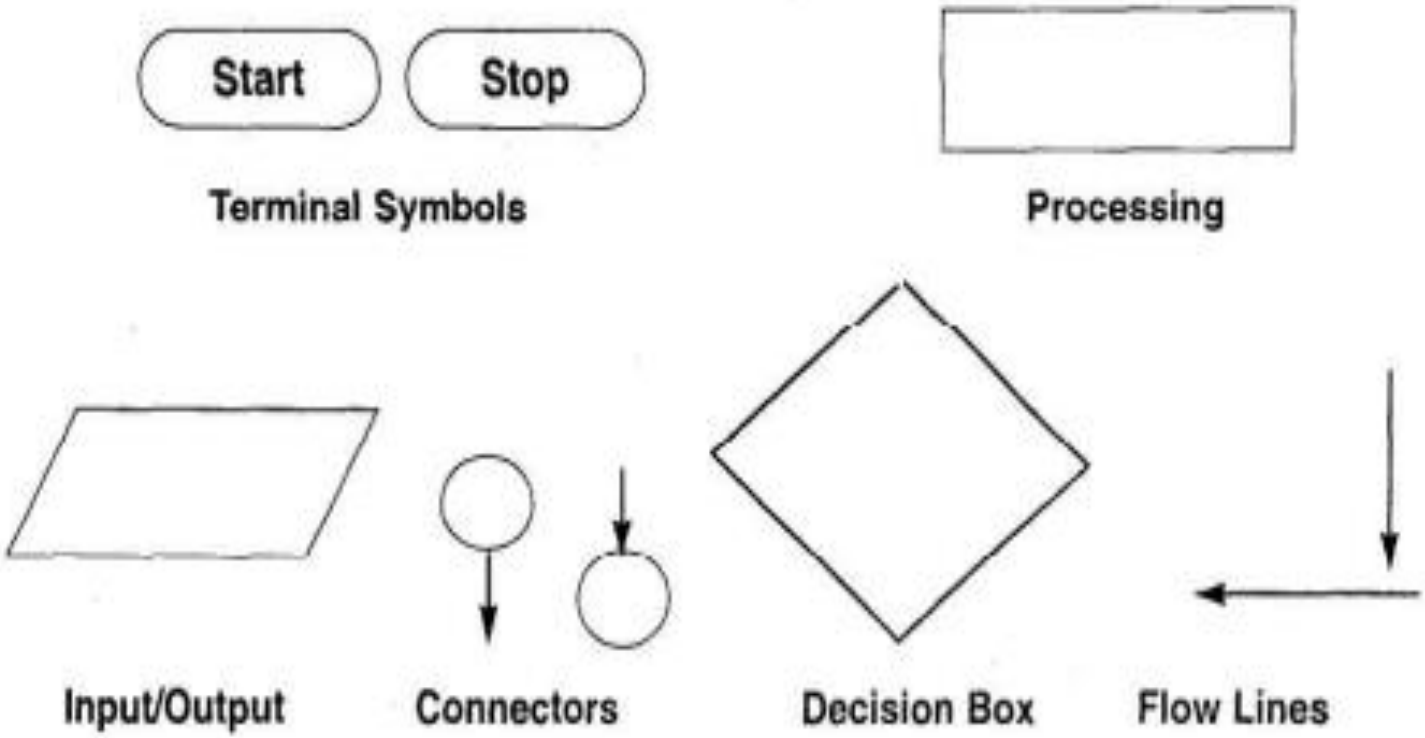


Fig. 1 : Flowchart Symbols

Chap. 1-2 algorithm PRESENTATION

- An algorithm has the following ***structures***;

–Input Step

–Assignment Step

–Decision Step

–Repetitive Step

–Output Step

Practical examples

In class the students must perform the exercises to upgrade their skills in Fundamentals of algorithms

- Given n numbers, then design an algorithm for finding: (simple mathematical operators)

a) The sum, difference , multiplication, division

b) power,

c) Modulo

d) The sequence of numbers,..

Cont'

- Given n numbers, then design an algorithm for finding: (Statistical parameters)

a) The sum
b) the average
c) The variation
d) The standard deviation
e) Median
f) The mode

CONT'

Rules for giving a valid name to a variable

- Start a variable name with a upper case letter
- **Don't use a blank** space in variable name
- Instead of using a blank space in a variable name **put underscore**
- Don't use reserved words
- Don't use special characters

Chap: 1-4 Categories of algorithm

- Based on the different types of steps in an Algorithm, it can be divided into four categories, namely
  1. Sequence
  2. Selection [Decision]
  3. Iteration [Loop]
  4. Recursion
- In the next slide, we discuss each in details

CHAP 1-4:1 SEQUENCE

- The sequence structure involves a chain of block of instructions.

Syntax

Instructions bloc(1)

Instructions bloc(2)

……

Instructions bloc(n)

Example1. Write a pseudocode and flowchart that:

a) simultaneously **add** and **substract** two numbers

b) Compute the **average** of an array of numbers

CHAP. 1-3 EXERCISES

Design an algorithm that computes:

1. Simultaneously return the sum, defference and multiplication of two numbers
2. The expression Z=(3x+4y)/4
3. The variance of 5 numbers
4. The net income given gross income and expenses
5. Net profit given capital investment , interest rate, and time
6. Covered distance given speed and time
7. Area of square given side of squarenet income is:

Chap. 1-3 Exercises

- Design an algorithm that computes:

1. Volume a cylinder given diameter of its base area
2. Circumference of circle given diameter d,
3. Area of rectangle triangle with base b, and height.
4. The number of attendance of students in each month of 24 days of class attendance.
5. The total cost of items , given the number of items ,Q, and the cost price of each item ,P
6. The marks' percentage given marks.

CHAP 1-4: 2. SELECTION

- The category structure is used to test a block of instructions. It has two main parts: the part which evaluate a condition, and a part of one instruction or a block of instructions.

- Conditions that are used in **decision control** structure are:
  - if
  - if… then… else
  - Nested Ifs
  - Switch

Syntax

If (condition) then

    Instructions

End if

- The if statement is used to make a decision. The block of instructions following the if executes if the decision is true, and the block does not execute otherwise

Example

Write a pseudocode and flowchart that:

a) Promote students who marks is grater than 50

CHAP 1-4: 2. SELECTION (IF..ELSE STATEMENT)

- The if…else statement is used to make a decision and gives the alternative when the condition evaluates to false.

- The block of instructions following the if executes if the decision is true, and the block **after else when the condition evaluates to false**

Example

Write a pseudocode and flowchart that:

a) Compute the median
b) Check if the given number is divisible by 11

CHAP 1-4: 2. SELECTION (NESTED IFS)

Nested Ifs

Nested If in constists of placing If Statement inside another IF Statement.

.

Nested if syntax

```
if expression1:
    statement(s)
        if  expression2:
            statement(s)
        else
            statement(s)
else:
        if expression3:
        statement(s)
        else:
            statement(s)
```

Cont'

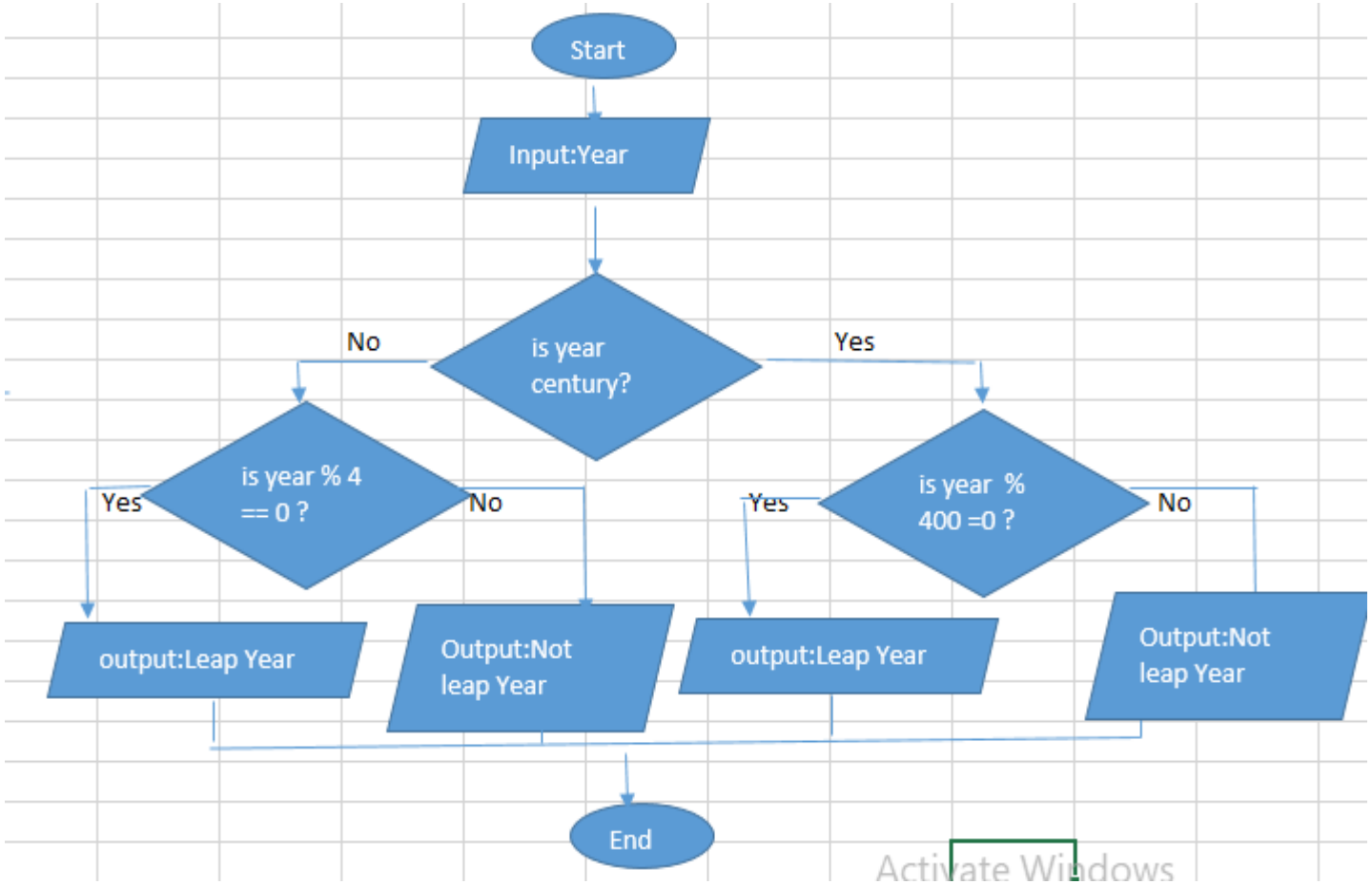
- Example1:
- Use the nested **if** statements to determine if a number is divisible by 6.

Hint(a number is divisible by 6 if and only if it is divisible by 2 and 3.

Example2: Check the leap year (See procedure in next slide)

Cont'

- Flowchart



Ladder if … else if statement

if else if ladder is an extension of if else statement used to sequentially test a series of conditions, executing the code for the first true condition.

Ladder if … else syntax

```
if (condition1) {
            // Statements 1
              }
else if (condition2) {
      // Statements 2
            }
…
else {
// Else body
}
```

Cont'

Answer:

```
Var day as integer
Start
Write("enter the day")
Read(day)
If (day= =1) then
Write(" Sunday")
Else if( day = =2)then
Write("Monday")
Else if( day = =3)then
Write("Tuesday")
Else if( day = =4)then
Write("Wednesday")
Else if( day = =5)then
Write("Thursday")
Else if( day = =6)then
Write("Friday")
Else if( day = =7)then
Write("Saturday")
Else
Write("Unknown Day")
End if
end
```

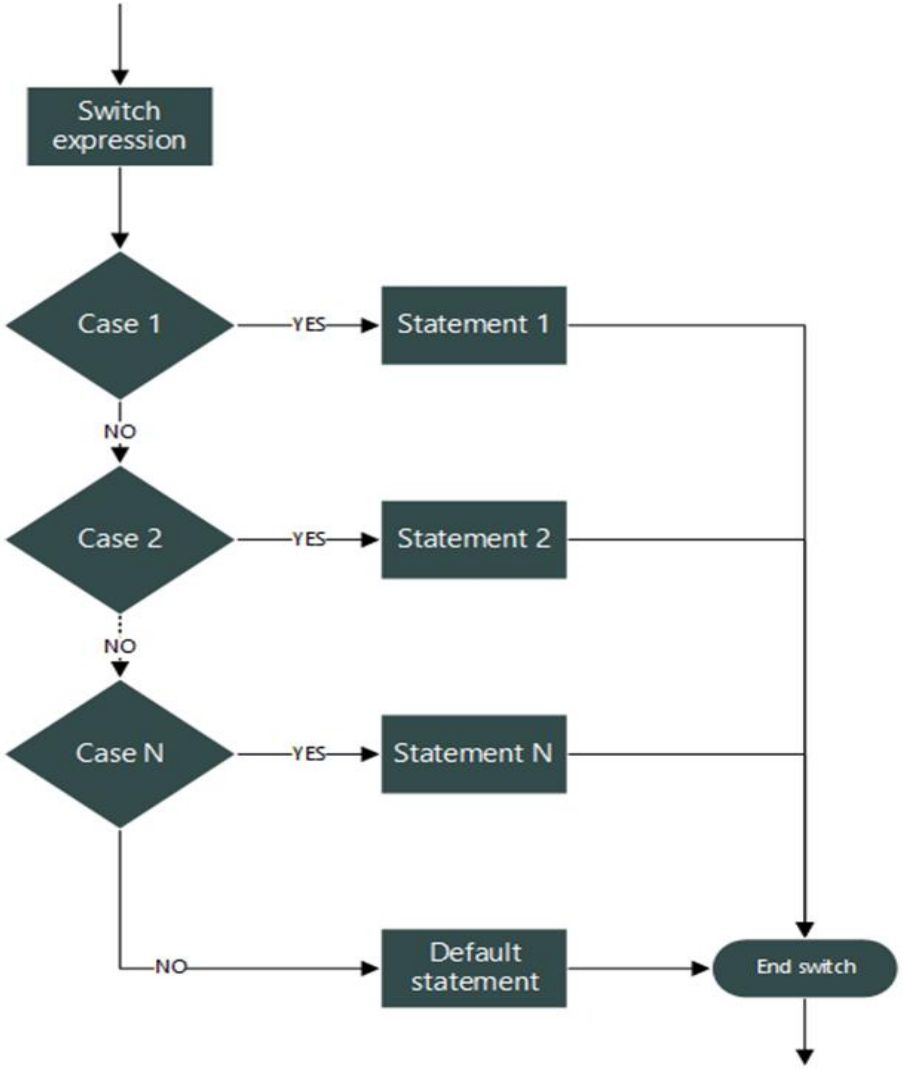
Multiple choice using 'switch'

A multiple choice using switch helps to solve the problem caused by nested if statement in case there are many conditions to be tested.

Switch receives a variable then it evaluates it using several **Case.**

CHAP 1-4: 2. SELECTION

Multiple choice using 'switch ' flowchart



CHAP 1-4: 2. SELECTION

Multiple choice using **'switch'**

Syntax:

Switch(variable)

Case 1

Instruction

Case 2

Instruction

….

Case n

Instruction

Default

Instruction

End switch

CHAP 1-4: 2. SELECTION

Example 3 a:

Write an algorithm which receives note and displays the student's grade.

16 and above: grade A

14-16: grade B

12-14: grade C

Less than 12: grade D

WRITE A PROGRAM THAT WILL RUN THE ABOVE INSTRUCTIONS

CHAP 1-4: 2. SELECTION

```
Var Note as integer
Start
Write("Use number to chose the range of your note")
Write("enter 1 for note ranging from 16 and above")
Write("enter 2 for note ranging from 14 to16")
Write("enter 3 for note ranging from 12 to14")
Write("enter 4 for note less than 12")
Write("enter your choice now using number:")
Read(Note)
Switch(Note)
Case 1
Write("You have grade A")
```

CHAP 1-4: 2. SELECTION

Example:

```
Case 2
Write("You have grade B")
Case 3
Write("You have grade C")
Case 4
Write("You have grade D")
Default
Write("Your choice is not listed, try again")
End switch
End
```

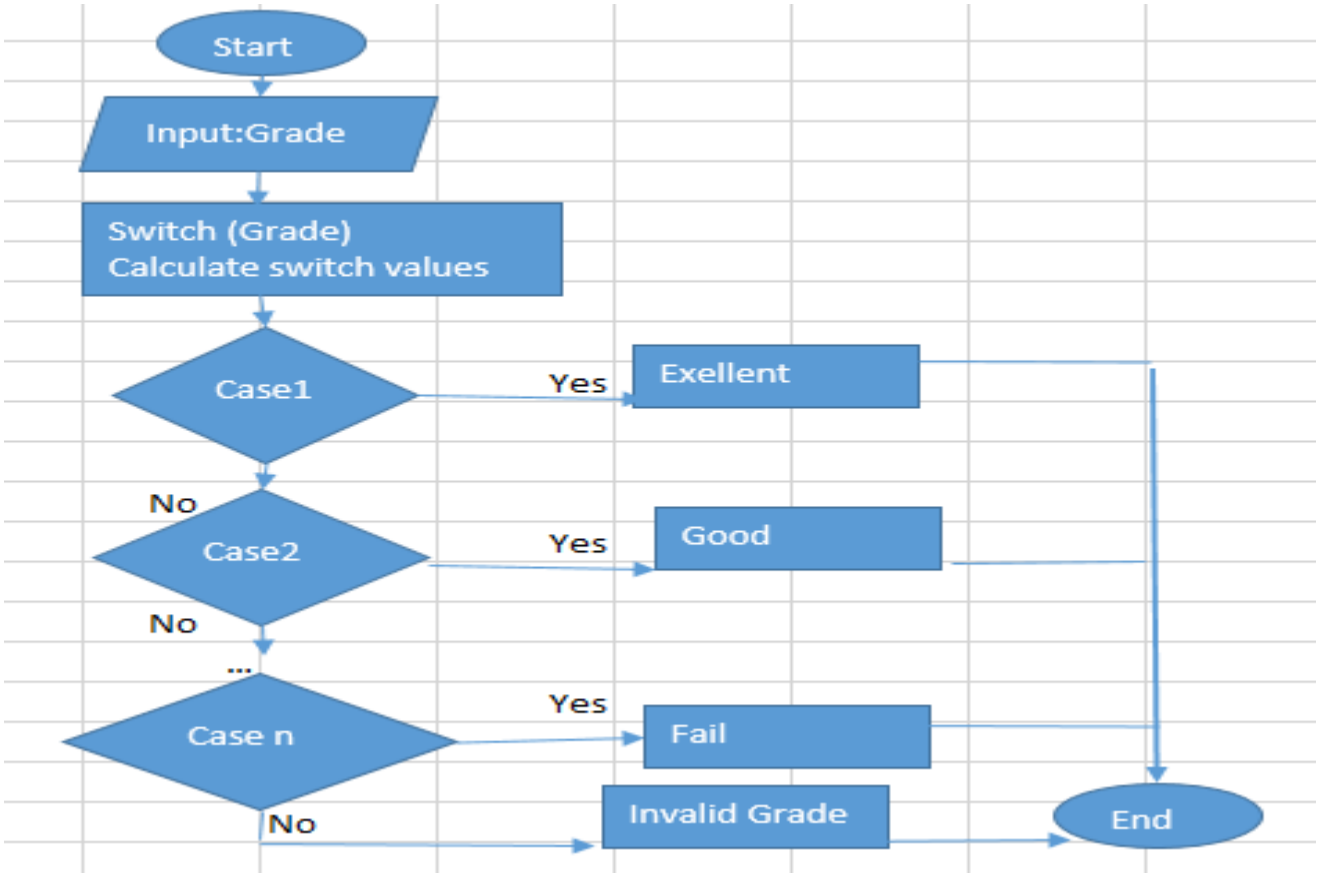
Cont'

- *Example No 3a: The grade of students marks are given, the using the following rules to design the flowchart , and then write a program in your favorite programming language*

Marks	Grade	
80-100	Excellent	
66-79	Very Good	
56-65	Good	
50-55	Weak	
30-49	Very Weak	
1-39	Very very Weak	

CHAP 1-4: 2. SELECTION

- Flowchart of grading system



CHAP 1-4: 3. ITERATION /LOOP

- What is a loop? A loop helps to repeat instruction or block of instructions. It assists in the algorithm where you want to carry out an activity for a certain number of times. An iteration structure represents part of the program that repeats. This type of structure is commonly known as a loop
- In Algorithm, loops are divided into 3 categories:

1. Loops (do while, until)

   a) Do while …loop

   b) Do until…loop

   c) Do… loop Until

   d) Do… loop while

- 2. Iterative Loops (For loop)
- 3. Loops in Loops

while ...loop

With the while loop we can execute a set of statements as long as a condition is true.

- **Example1:** Write an algorithm which use do while loop and displays numbers from 10 to 15
- **Answer:**

```
Var i as integer
Start
  i ← 10
  Do while i<=15
    Write(i)
  i ← i+1
   Loop
End
```

Cont

- Example2:
- Write an algorithm, flowchart/ Program :

1. to add numbers until the user enters zero
2. To count the number of digits in a given number.

Cont'

- While loop may be used with some key word like
- Break : to stop the loop
- Continue :to skip an outcome
- Else : to add other condition

CHAP 1-4: 3. ITERATION /LOOP

- **Do until…loop**

Example

Write ana algorithm which displays the Numbers between 1 to 100 will be displayed on the form as soon as you click on the command button.

```
Variable Num As integer
Num ← 0
Do Until Num < 100
Num = Num + 1
write (Num)
Loop
```

CHAP 1-4: 3. ITERATION /LOOP

- **Do… loop while**

The do-While loop execute its statements at least once even if the condition fails for the first time.

Syntax:

<Variable>=<start value>

Do

Instruction or block of instructions

Variable=variable+1

Loop while

<variable><comparison

operator><end value>

CHAP 1-4: 3. ITERATION /LOOP

- **Do… loop Until**

Executes the <statements> first and then tests the <condition>.

Loop is executed at least once and then tests the condition and Executes till condition is false.

Example: Write an algorithm which use do while loop and displays numbers from 10 to 20

Answer: Var A as integer

```
Start
  A=10
  Do
     Write(A)
     A=A+1
  Loop until A<=20
End
```

- **2. Iterative Loops**

For loop

- The for loop is an iterative loop, it specifies some elements about the loop in one single line Setting a loop counter to an initial value. End which determine whether its value has reached the number of repetitions desired

- The value of the loop counter will be increased each time (iteration), and segment within the loop will be executed.

Syntax:

for (<initialize counter> to <end of repetitions desired>) do
Instruction or bloc of instructions
end for

- **Iterative Loops**

- **Example:** Write an algorithm which ask a user to enter a number and it displays the 20 next numbers.
- Var I,A as integer

```
Start
  Write("enter a number")
   Read(A)
  For (I=A to I+20) do
    Write(I)
   I ← I+1
  End for
End
```

- **3. Loops in Loops**

- Loops in loops refer to what we call nested loops.
- These are loops that are such that when one increment by one the other continues inside the first.
- A nested loop is when you have one loop inside another

- The general structure of a nested For… loop is:

For……’start of outer loop

For…….’start of inner loop

…….’body of inner loop

End for ……’end of inner loop

End for……..’end of outer loop

Cont'

- Example1:
- Write an algorithm that can create a loop to iterate four times (4 weeks). And, inside the loop, we can create another loop to iterate 7 times (7 days).
- Use while loop as outer loop,
- then for loop as inner loop.

- Example2:
- Write an algorithm that create a half pyramid pattern using nested loops.

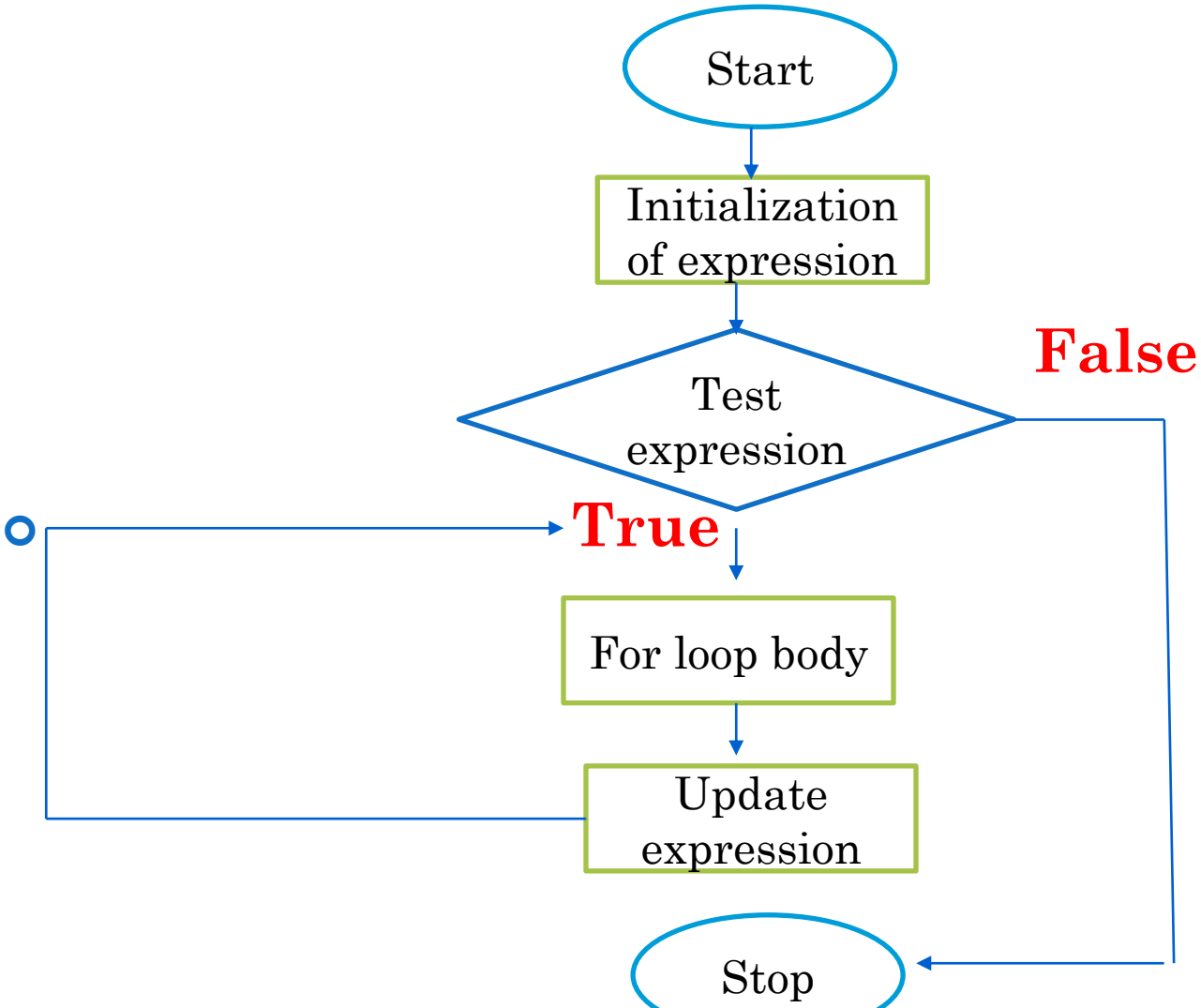
Cont'

- Like simple for loop, the nested for loops may contains:
- **Break and continue** inside the

- When we use a break statement inside the inner loop, it terminates the inner loop but not the outer loop.

- Example: Write an algorithm that print a list of your students in seven days by breaking day six and seven .{Assignment No 1)[Pseudocodes +Flowchart + codes]

FOR LOOP FLOWCHART



Cont'

- Example :
- Write an algorithm to calculate :

1. The sum of first n natural numbers

- // Positive integers 1,2,3...n are known as natural numbers

2. The factorial of a non-negative integer n

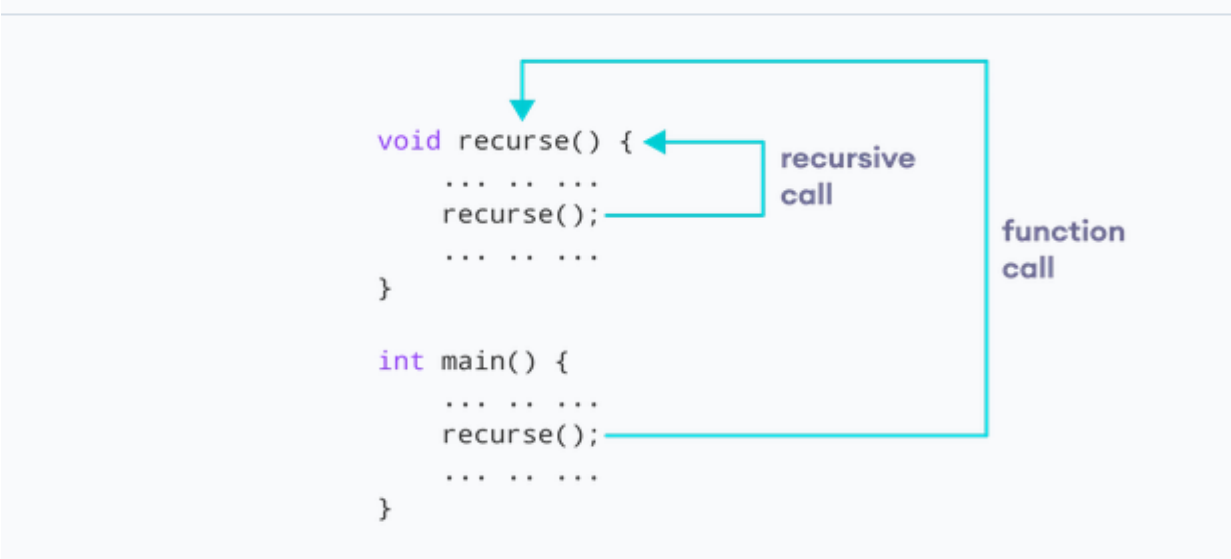
CONT

- **Infinite while Loop**
- If the condition of a while loop always evaluates to True, the loop runs continuously, forming an infinite while loop.

- fees = 10$
- # The test condition is always True
- while fees > 5$:
- print('You can sit for exam')

UNIT I-5: RECURSION

- A recursive function is a type of function that calls itself . And, this technique is known as recursion.
- **In programming, a recursion** is a method of solving a computational problem where the solution depends on solutions to smaller instances of the same problem.



UNIT I-5-1: RECURSION TYPES

- The recursion method is classified as follows :
- **Direct recursion:** The function calls **itself directly** to accomplish a task.

Example

```
algorithm findFactorial(number):
  // INPUT
//   number = A positive integer
  // OUTPUT
Returns the factorial of the number

 if number = 1
     return 1
Else
return number * findFactorial(number - 1)
```

UNIT I-5-1: RECURSION TYPES

- **Indirect recursion:** Two functions **call each other repeatedly** to accomplish their respective tasks.
- In indirect recursion, a function calls another function which then calls the first function again[Contrapositive in Logic}
- Example,
- we ***may define a function to check if the given positive number is even by using the odd function .***
- ***The same the function that detects if the function is odd by using the even function.***

Unit I-5-1: Recursion types

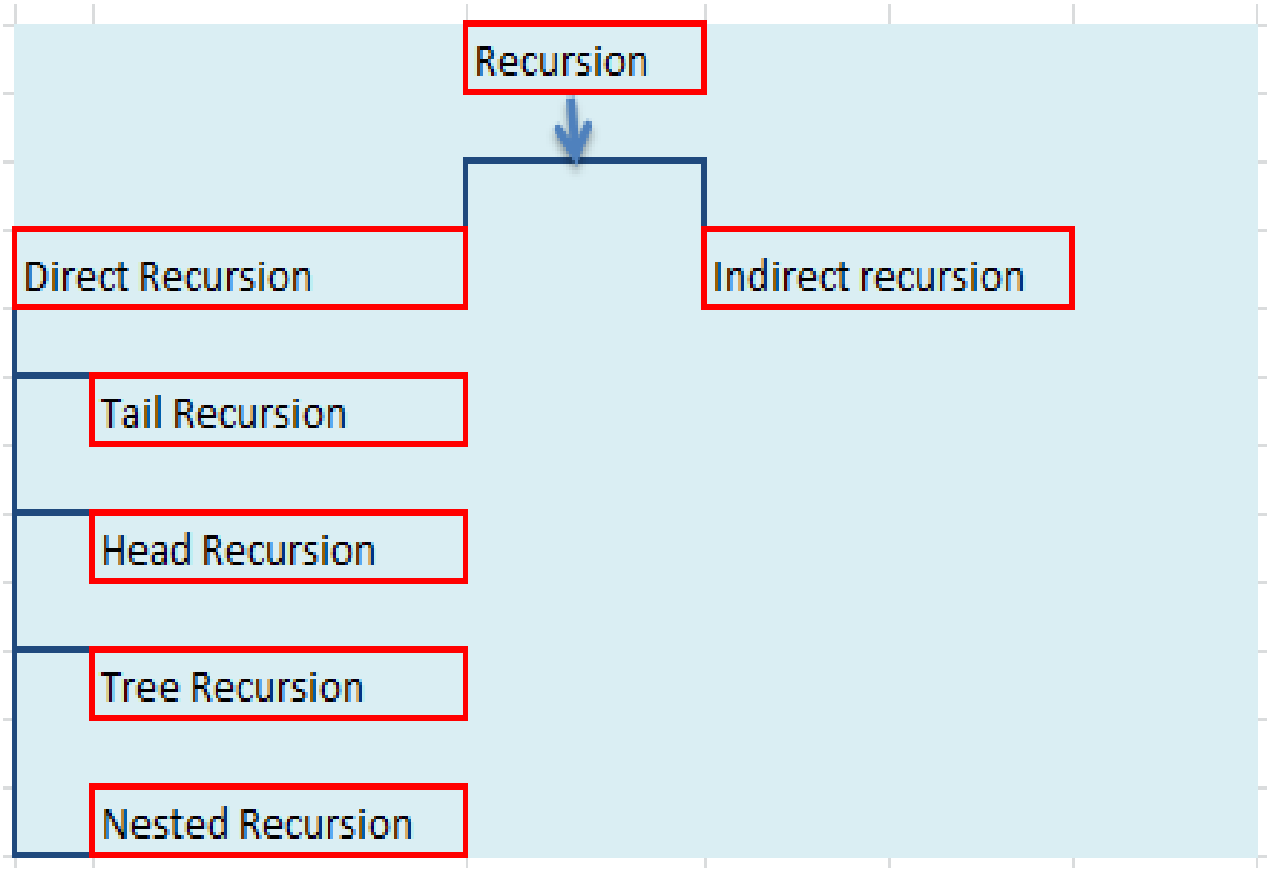
- Example

```
Algorithm Is_Even (n):
    for n positive:
    if n=0:
     return true
   else:
      return isOdd (n-1)

Algorithm Is_odd (n):
    for n positive:
    if n=0:
     return odd
   else:
      return isEven (n-1)
```

Unit I-5-1: Recursion types

- Graph that illustrates Recursion



UNIT I-5-1: RECURSION TYPES

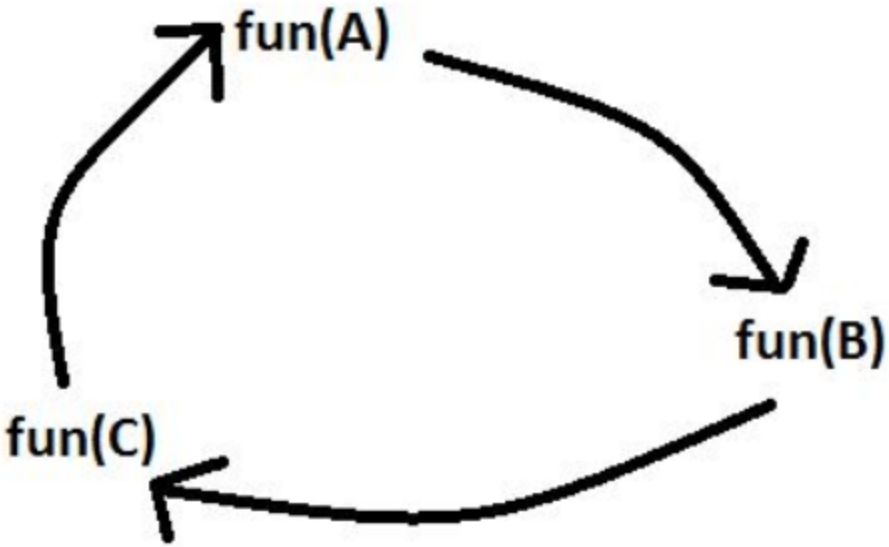
- **1-1:Tail Recursion : It is direct recursion whereby** a recursive function calling itself and that recursive call is **the last before running the results. (Display function comes after)**
- Example : tail_factorial(n - 1, acc * n) for tail_factorial(n, acc=1):
- **1-2: Head Recursion: It is direct recursion that** a recursive function calling itself and that recursive call is the first statement in the function then it's known as **Head Recursion.** There's no statement, no operation before the call. (Display function comes before)
- head_recursion(n - 1) for head_recursion(n - 1)

UNIT I-5-1: RECURSION TYPES

- **1-3:Tree Recursion:** If a recursive function calling itself for one time then it's known as **Linear Recursion**. Otherwise if a recursive function calling itself for more than one time then it's known as **Tree Example :Fibonacci function**
- **1-4:Nested Recursion:** In this recursion, a recursive function will pass the parameter as a recursive call. That means **"recursion inside recursion".**

UNIT I-5-1: RECURSION TYPES

- **2. Indirect Recursion or mutual recursion** : In this recursion, there may be more than one functions and they are calling one another in a circular manner. Example calling Even Fxn by using Odd function



Unit I-5-2: Recursive data types

- Inductively Data type
- It specifies how to construct instances of the data. For example, linked list can be defined inductively
- Coinductive data definition
- It specifies the operations that may be performed on a piece of data;

Unit I-5-3: Parts of a Recursive Algorithm

- Example: Adding three numbers is equivalent to adding the first two numbers, and then adding these two numbers again.
- Use the uniform three stages:

1. Base Case: if (number of parameters == 2)

   result = a + b;

2. "Work toward base case":

   a+b becomes the first parameter

This reduces the number of parameters sent in to the function from 3 to 2, and 2 is the base case!

3. Recursive Call: add_numbers(a+b, c);

Unit V-3-3: Parts of a Recursive Algorithm

All recursive algorithms must have the following:

- Base Case (i.e., when to stop)
- Work toward Base Case
- Recursive Call (i.e., call ourselves)

- Base Case
- A recursive function definition has one or more *base cases*, meaning input(s) for which the function produces a result trivial (without recurring), and one or more *recursive cases*, meaning input(s) for which the program recurs (calls itself)
- Example :Factorial
- If n= $0! = 1$ and, for all $n > 0$, $n! = n(n - 1)!$.
- Recursive cases can be seen as breaking down complex inputs into simpler ones.

- For Factorial base case is if n==0, and return 1

```
factorial(5)
= 5 * factorial(4)
= 5 * 4 * factorial(3)
= 5 * 4 * 3 * factorial(2)
= 5 * 4 * 3 * 2 * factorial(1)
= 5 * 4 * 3 * 2 * 1 * factorial(0)
= 5 * 4 * 3 * 2 * 1 * 1   (base case returns 1)
= 120
```

EXAMPLE2

- Example :Binary search
- Define Recur_Bi_Search (Array, lb, ub, Target)
- Compute M=(lb+ub)/2
- Test (if A[M]==Target)
- If Yes target found
- If if A[M]<Target
- Display Recur_Bi_Search (Array, M+1, ub, Target)
- Else:
- Display Recur_Bi_Search (Array, lb, M-1, Target)

end If

Display no value

Unit I-5-4: Recursion Applications

1. Summing a list of numbers:

- What is a recursive solution to summing up a list of numbers? First you should note that the sum of [1 2 3 4 5 6 7 8]) is equal to 1 + sum of [2 3 4 5 6 7 8])!
- Syntax: result = list(1) + sum (list(2:end));

Sum([ ])=0 ←Base Case (empty list)

Sum([X]+rest)=X+ Sum(rest) ←Recursive Case

Unit I-5-4: Recursion Applications

- Fibonacci Function
- What is the definition of the Fibonacci number at location X in the sequence?
- Syntax: fib(x) = fib(x-1) + fib(x-2);

```
F(0) = 0          ← base case 1

F(1) = 1          ← base case 2

F(n) = F(n-1) + F(n-2)   ← recursive case for n > 1
```

Unit I-5-4: Recursion Applications

5. Sorting

- How can you sort a list of numbers using recursion?
- Hint 1: Is it easier to sort a long list or a short list?
- Hint 2: If you have two sorted lists, can you put them back together?
- Syntax: list3 = merge (sorted list1, sorted list2), whereby list1 and list2 are partitions of the given list.

6.Triangle of Pascal

UNIT I-6: ALGORITHMS ANALYSIS

- The process of evaluating the performance of an algorithms, by calculating , **order of growth** of time taken (or space) by an algorithm in terms of input size.
- This process is known as Asymptotic analysis that consists of evaluating the algorithm in terms of input size
- **Types of Algorithm Analysis:**

1. Best case
2. Worst case
3. Average case [Details in next slides]

UNIT I-6-1: ALGORITHMS ANALYSIS IMPORTANCE

- **Performance analysis is crucial because**

1. **To provide the functionality of system in terms of (**like user-friendliness, modularity, security, maintainability)
2. To summarize, performance == scale the use needs to preforms task.

Example: A system that can that can load 100 pages, but can spell check 1 page per minute.

Unit I-6-1: Algorithms Analysis Importance

- **For example2.,** let us consider the search problem (searching a given item) in a sorted array.
- The solution to above search problem includes:
- **Linear Search** (**order of growth** is **linear**)
- **Binary Search** (**order of growth** is logarithmic).

Order of growth

- Let f(n) and g(n) be the time taken by two algorithms where n >= 9 and f(n) and g(n) are also greater than equal to 0.
- A function **f(n)** is said to **be growing faster than g(n)** if **g(n)/f(n)** for n tends to infinity is 0 (or f(n)/g(n) for n tends to infinity is infinity).

- **Example 1:** f(n) = 100, g(n) = n + 1
  For n > 99, g(n) would always be greater than f(n) because order of growth of g(n) is more than f(n).

- **Example 2**: f(n) = $4n^2$, g(n) = 2n + 13
  f(n) has higher order of growth as it grows quadratically in terms of input size.

Quickly find order of Growth?

- When n >= 0, f(n) >= 0 and g(n) >= 0, we can use the below steps.
- Ignore the lower order terms.
- Ignore the constants
- **Example 1 :** $4n^2 + 3n + 100$
  Ignore lower order terms, we get $4n^2$
  After ignoring constants, we get n^2
  Hence order of growth is n^2

Exampl2: 100 n Log n + 3n + 100 Log n + 2

After ignoring lower order terms, we get 100 n Log n

After ignoring constants, we get **n Log n**

Hence order of growth is **n Log n**

Unit I-6-1: Algorithms Analysis Importance

- We run the Linear Search on a fast computer A and Binary Search on a slow computer B as follows:

Input Size	Running time on A	Running time on B	Decision
10	2 sec	~ 1 h	Time A is less time B
100	20 sec	~ 1.8 h	Time A is less time B
10^6	~ 55.5 h	~ 5.5 h	Time A is greater time B
10^9	~ 6.3 years	~ 8.3 h	Time A is less time B

UNIT I-6-1: ALGORITHMS ANALYSIS IMPORTANCE

- · For small values of input, the fast computer A may take less time than B.
- · But, after a certain value of input array size, the Binary Search will definitely start taking less time compared to the Linear Search even though the Binary Search is being run on a slow machine.

Reasons:

- The machine specific factors would not matter as the value of input would become large.
- ·The order of growth of Binary Search with respect to input size is **logarithmic** (Exponentially) while the order of growth of **Linear Search is linear.**

Unit I-6-1: Algorithms Analysis Importance

- Let 0.2 be a constant for machine A and 1000 the constant for B , then
- Running times for this example:
- Linear Search running time in seconds on A: 0.2 * n
- Binary Search running time in seconds on B: 1000*log(n)

Unit I-6-1: Algorithms Analysis Importance

- **complexity for single for-loop**

```
a = 0
b = 0
N = 5
M = 5

# This loop runs for N time
for i in range(N):
    a = a + 10
# This loop runs for M time
for i in range(M):
    b = b + 40

print(a, b)
```

Answer: 50, 200

Comparing order growth

- The Time complexity here will be ***O(N + M)***. Loop one is a single for-loop that runs **N times** and calculation inside it takes ***O(1) time***. Similarly, another loop takes **M times** by combining both the different loops takes by adding them is ***O(N + M + 1) = O(N + M)***.

- **Some standard terms**

$c < \text{Log Log } n < \text{Log } n < n^{1/3} < n^{1/2} < n < n \text{ Log } n < n^2 < n^2 \text{ Log } n < n^3 < n^4 < 2^n < n^n$

Unit I-6-1: Algorithms Analysis Importance

- Given an algorithm that returns the response by updating i=i/2 in each iteration , then

```
While i>1:
    i=i/2
```

At each iteration the value of i is halved:

Iteration 1: i = n / 2

Iteration 2: i = n / 4

Iteration 3: i = n / 8

..

Iteration k: i = n / 2^k

Unit I-6-1: Algorithms Analysis Importance

- .

We stop i<1. we solve for i=n

$= (n/2^K <= 1$ (for k iterations)

$= 2^{k >= n}$ (taking log on both sides)

$= k >= \log(n)$ base 2.

Therefore, the time complexity will be

$T(N) = O(\log N)$

UNIT I-6-2: ALGORITHMS ANALYSIS TYPES

1) Best case: Define the input for which algorithm takes less time or minimum time.

- In the best case calculate the lower bound of an algorithm.
- Example: In the linear search when search data is present at the first location of large data then the best case occurs.

2) Worst Case: Define the input for which algorithm takes a long time or maximum time. In the worst calculate the upper bound of an algorithm.

- Example: In the linear search when search data is not present at all then the worst case occurs.

UNIT I-6-2: ALGORITHMS ANALYSIS TYPES

3) Average case: In the average case take all random inputs and calculate the computation time for all inputs. And then we divide it by the total number of inputs.

Average case = all random case time / total no of case

UNIT I-6-3: ALGORITHMS ANALYSIS NOTATION

- The three most commonly used analysis notations are : Big O, Omega, and Theta. Time complexity estimates the time to run an algorithm. Its based on algorithm analysis type

Notation	Types	Functions Behavior	Time
Omega Notation	**best case**	functions grows Faster/	minimum amount of time an algorithm requires to consider all input values.
Big-O Notation	**worst-case**	functions grows slower	Maximum amount of time an algorithm requires to consider all input values.
Theta Notation	Average case	functions lies in both **O(n)** and **Omega(n)**,	a time complexity average case for an algorithm.

Common Big O Notations (Time Complexity)

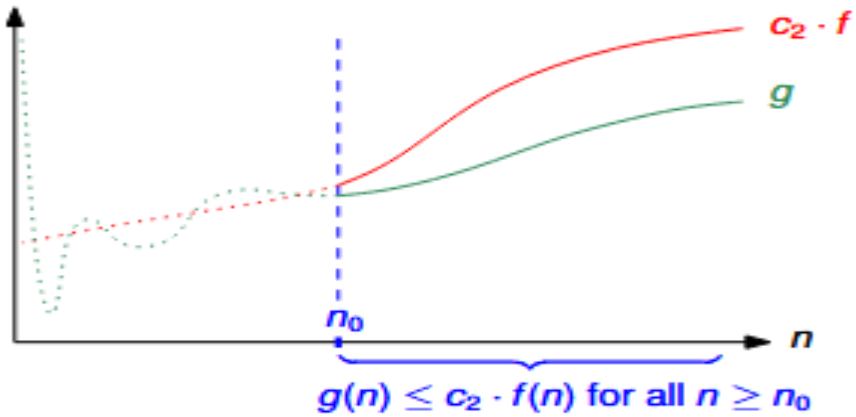
Big O	Name	Example Problem	Description
O(1)	Constant Time	Accessing an array element	Does not grow with input size
O(log n)	Logarithmic Time	Binary search	Input size halves each step
O(n)	Linear Time	Iterating through a list	Grows directly with n
O(n log n)	Linearithmic	Merge sort, quicksort (avg)	Efficient divide-and-conquer
$O(n^2)$	Quadratic Time	Bubble sort, nested loops	Grows fast as n^2
$O(2^n)$	Exponential Time	Recursive Fibonacci	Grows very fast
O(n!)	Factorial Time	Traveling salesman brute-force	Extremely inefficient for large n

Unit I-6-3: Algorithms Analysis Notation

Asymptotic Notation: Big-O

Let $f: \mathbb{N} \to \mathbb{R}^+$. Then the set $O(f)$ is defined as

$$O(f) := \{g: \mathbb{N} \to \mathbb{R}^+ \mid \exists c_2 \in \mathbb{R}^+ \quad \exists n_0 \in \mathbb{N} \quad \forall n \geq n_0 \qquad g(n) \leq c_2 \cdot f(n)\}.$$



- Equivalent definition used by some authors:

$$O(f) := \left\{g: \mathbb{N} \to \mathbb{R}^+ \mid \exists c_2 \in \mathbb{R}^+ \quad \exists n_0 \in \mathbb{N} \quad \forall n \geq n_0 \qquad \frac{g(n)}{f(n)} \leq c_2\right\}.$$

- Some authors prefer to use the symbol $\mathcal{O}$ instead of O.

Asymptotic Notation: Big-O

- Example , the highest degree of f(n) is n squared

Let:

- $f(n) = n^2$
- $g(n) = 3n^2 + 5n + 7$

We want to **prove** that:

$$g(n) \in O(f(n)) \quad \text{or} \quad 3n^2 + 5n + 7 \in O(n^2)$$

We must find **constants** $c_2 > 0$ and $n_0 \in \mathbb{N}$ such that:

$$g(n) \leq c_2 \cdot f(n) \quad \text{for all } n \geq n_0$$

i.e.

$$3n^2 + 5n + 7 \leq c_2 \cdot n^2 \quad \text{for all } n \geq n_0$$

Asymptotic Notation: Big-O

- So divide f(n) on both side, and take largest value of n .

Divide both sides by n^2:

$$\frac{3n^2 + 5n + 7}{n^2} \leq c_2 \Rightarrow 3 + \frac{5}{n} + \frac{7}{n^2} \leq c_2$$

As $n \to \infty$, the extra terms shrink:

$$\frac{5}{n} \to 0, \quad \frac{7}{n^2} \to 0 \Rightarrow \text{So the LHS approaches } 3$$

Asymptotic Notation: Big-O

Let's try $c_2 = 4$. Then:

$$3 + \frac{5}{n} + \frac{7}{n^2} \leq 4 \Rightarrow \frac{5}{n} + \frac{7}{n^2} \leq 1$$

Let's test for which n this h

Try $n = 5$:

- $c_2 = 4$
- $n_0 = 10$

Thus,

$$3n^2 + 5n + 7 \leq 4n^2 \quad \text{for all } n \geq 10 \Rightarrow g(n) \in O(n^2)$$

Try $n = 10$:

$$\frac{5}{10} + \frac{7}{100} = 0.5 + 0.07 = 0.57 \quad \checkmark$$

So for $n \geq 10$, the inequality holds.

- Results: C2=4 and no=10;

$$3n^2 + 5n + 7 \leq 4n^2 \quad \text{for all } n \geq 10 \Rightarrow g(n) \in O(n^2)$$

$$3n^2 + 5n + 7 \in O(n^2)$$

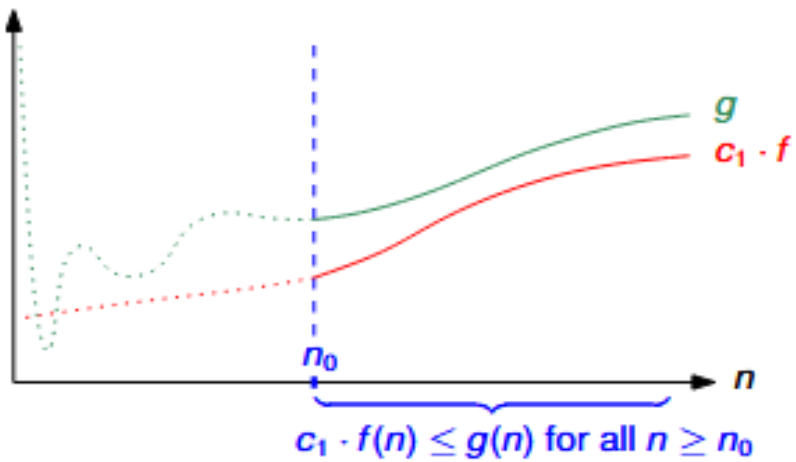
This confirms that $g(n)$ is upper-bounded by n^2 asymptotically.

Unit I-6-3: Algorithms Analysis Notation

- Asymptotic Notation: Big-Omega

Let $f: \mathbb{N} \to \mathbb{R}^+$. Then the set $\Omega(f)$ is defined as

$$\Omega(f) := \{g: \mathbb{N} \to \mathbb{R}^+ \mid \exists c_1 \in \mathbb{R}^+ \quad \exists n_0 \in \mathbb{N} \quad \forall n \geq n_0 \quad \boxed{c_1 \cdot f(n) \leq g(n)}\}.$$



- Equivalently,

$$\Omega(f) := \left\{g: \mathbb{N} \to \mathbb{R}^+ \mid \exists c_1 \in \mathbb{R}^+ \quad \exists n_0 \in \mathbb{N} \quad \forall n \geq n_0 \quad c_1 \leq \frac{g(n)}{f(n)}\right\}.$$

Big Omega (Ω) – Lower Bound

A function $g(n) \in \Omega(f(n))$ if:

$$\exists\, c_1 > 0,\ n_0 \in \mathbb{N},\ \text{such that}\ g(n) \geq c_1 \cdot f(n)\ \ \forall n \geq n_0$$

We want to show:

$$3n^2 + 5n + 7 \geq c_1 \cdot n^2 \Rightarrow \frac{3n^2 + 5n + 7}{n^2} \geq c_1 \Rightarrow 3 + \frac{5}{n} + \frac{7}{n^2} \geq c_1$$

As $n \to \infty$, the extra terms go to 0, so LHS → 3.

- Let's try $c_1 = 2$. Then:

$$3 + \frac{5}{n} + \frac{7}{n^2} \geq 2 \Rightarrow \text{Always true for large } n$$

$n = \infty$

Try $n = 1$:

$$3 + 5 + 7 = 15 \geq 2 \quad \checkmark$$

Try $n = 5$:

$$3 + 1 + 0.28 = 4.28 \geq 2 \quad \checkmark$$

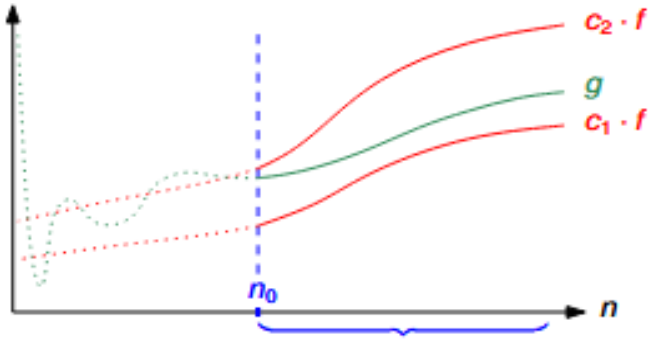
So, for all $n \geq 1$, this holds.

Unit I-6-3: Algorithms Analysis Notation

- Asymptotic Notation: **Big Theta (Θ)** – Tight Bound

Let $f: \mathbb{N} \to \mathbb{R}^+$. Then the set $\Theta(f)$ is defined as

$$\Theta(f) := \{g: \mathbb{N} \to \mathbb{R}^+ \mid \exists c_1, c_2 \in \mathbb{R}^+ \quad \exists n_0 \in \mathbb{N} \quad \forall n \geq n_0 \quad c_1 \cdot f(n) \leq g(n) \leq c_2 \cdot f(n)\}.$$



$c_1 \cdot f(n) \leq g(n) \leq c_2 \cdot f(n)$ for all $n \geq n_0$

which is equivalent to $c_1 \leq \frac{g(n)}{f(n)} \leq c_2$ for all $n \geq n_0$

Algorithms Analysis Notation

- By using the above example ;

$$g(n) \in \Theta(f(n)) \quad \text{if} \quad g(n) \in O(f(n)) \text{ and } g(n) \in \Omega(f(n))$$

We already showed:

- $g(n) \leq 4n^2$ for $n \geq 10 \Rightarrow O(n^2)$
- $g(n) \geq 2n^2$ for $n \geq 1 \Rightarrow \Omega(n^2)$

So both bounds hold!

Programming language

- I this class we have to use Python as programming Language .
- Use this link to get it https://www.python.org/downloads/windows/